EDGE INTELLGENCE LAB – 6

-Ganesh G(25MML0043)

Car Number Plate Detection

[2]:
```
import tensorflow as tf
from tensorflow.keras import layers, models


DATA_DIR = "/kaggle/input/car-number-plate-detection"

# Load the training dataset
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size = (160, 160),
    batch_size=32
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(160, 160),
    batch_size=32
)
```

```
val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(160, 160),
    batch_size=32
)

for images, labels in train_ds.take(1):
    print("Batch image shape:", images.shape)
    print("Batch label shape:", labels.shape)
```

```
Found 926 files belonging to 1 classes.
Using 741 files for training.
Found 926 files belonging to 1 classes.
Using 185 files for validation.
Batch image shape: (32, 160, 160, 3)
Batch label shape: (32,)
```

[4]:
```
# Standardize data (Rescale pixel values to [0, 1])
normalization_layer = layers.Rescaling(1./255)
train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))
```

[6]:
```
import tensorflow as tf
from tensorflow.keras import layers, models

# Define the number of classes (e.g., 36 for 0-9 and A-Z)
num_classes = 36

def build_cnn_model(input_shape=(160, 160, 3), num_classes=36):
    model = models.Sequential([
        # Initial Convolutional block
        layers.Conv2D(32, (3, 3), activation='relu', input_shape=input_shape),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        # Second Convolutional block
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        # Third Convolutional block
        layers.Conv2D(128, (3, 3), activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),
```

```
        # Third Convolutional block
        layers.Conv2D(128, (3, 3), activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        # Flatten and Dense layers
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.5),  # Regularization to prevent overfitting
        layers.Dense(num_classes, activation='softmax') # Output layer
    ])

    return model

# Create and compile the model
model = build_cnn_model()

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Display the model architecture
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 158, 158, 32)	896
batch_normalization_3 (BatchNormalization)	(None, 158, 158, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 79, 79, 32)	0
conv2d_4 (Conv2D)	(None, 77, 77, 64)	18,496
batch_normalization_4 (BatchNormalization)	(None, 77, 77, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 38, 38, 64)	0
conv2d_5 (Conv2D)	(None, 36, 36, 128)	73,856
batch_normalization_5 (BatchNormalization)	(None, 36, 36, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 18, 18, 128)	0
flatten_1 (Flatten)	(None, 41472)	0
dense_2 (Dense)	(None, 128)	5,308,544
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 36)	4,644

Total params: 5,407,332 (20.63 MB)

Total params: 5,407,332 (20.63 MB)

[7]:

```
# Training the model
EPOCHS = 20  # Adjusted for typical dataset convergence in 2026
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)

# Save the trained model for inference
model.save("number_plate_cnn_2026.keras")
```

```
Epoch 1/20
24/24 ━━━━━━━━━━━━━━━━━━━━ 71s 3s/step - accuracy: 0.8470 - loss: 1.3975 - val_accuracy: 0.0000e+00 - val_loss: 1
1.5645
Epoch 2/20
24/24 ━━━━━━━━━━━━━━━━━━━━ 65s 3s/step - accuracy: 0.9886 - loss: 0.2769 - val_accuracy: 0.0000e+00 - val_loss: 1
9.9241
Epoch 3/20
24/24 ━━━━━━━━━━━━━━━━━━━━ 66s 3s/step - accuracy: 0.9950 - loss: 0.0187 - val_accuracy: 0.0000e+00 - val_loss: 3
4.4289
Epoch 4/20
24/24 ━━━━━━━━━━━━━━━━━━━━ 67s 3s/step - accuracy: 0.9944 - loss: 0.0830 - val_accuracy: 0.0000e+00 - val_loss: 4
3.0780
Epoch 5/20
24/24 ━━━━━━━━━━━━━━━━━━━━ 65s 3s/step - accuracy: 0.9983 - loss: 0.0069 - val_accuracy: 0.0000e+00 - val_loss: 6
7.2296
Epoch 6/20
24/24 ━━━━━━━━━━━━━━━━━━━━ 65s 3s/step - accuracy: 0.9997 - loss: 7.8099e-04 - val_accuracy: 0.0000e+00 - val_los
s: 97.0111
Epoch 7/20
18/24 ━━━━━━━━━━━━━━━━━━━━ 16s 3s/step - accuracy: 0.9957 - loss: 0.0074
```